EXPERIMENT NO. 01
Star and Snowflake Schema

AIM
To write a detailed problem statement and design a dimensional model (creation of Star and Snowflake schema).

RESOURCES REQUIRED
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine.

THEORY

Problem Statement

All Electronics wants to create a sales data warehouse to keep records of the store's sales with respect to the dimensions time, item, branch and location. These dimensions allow the store to keep track of things like the monthly sales of items, and the branches and locations at which the items were sold.

Star Schema
A star schema is the simplest form of a dimensional model, in which data is organized into facts and dimensions.

- **Fact:** an event that is counted or measured, such as a sale or a log-in.
- **Dimension:** reference data about the fact, such as date, item or customer.

It is called a star schema because its entity-relationship diagram resembles a star, with points (dimension tables) radiating from a central table (the fact table).

Characteristics of Star Schema
1. Creates a de-normalized database that can quickly provide query responses.
2. Provides a flexible design that can be changed or extended easily throughout the development cycle, and as the database grows.
3. Mirrors the way end-users typically think of and use the data.
4. Reduces the complexity of metadata for both developers and end-users.
5. Every dimension is represented by exactly one dimension table, which contains the set of attributes. Each dimension table is joined to the fact table using a foreign key.
6. Easy to understand, needs fewer joins, and is widely supported by BI tools.
7. **Drawback:** more disk space and data redundancy, because the dimension tables are not normalized.

Snowflake Schema
A snowflake schema is an extension of the star schema. A schema is known as a snowflake if one or more dimension tables do not connect directly to the fact table, but must be joined through other dimension tables. Each point of the star "explodes" into more points, and the diagram resembles a snowflake.

The fact table is surrounded by its associated dimensions, and those dimensions are in turn related to other (sub-) dimensions, branching out in a snowflake pattern.

Characteristics of Snowflake Schema
1. **Main benefit:** uses less disk space, since the dimension tables are normalized and redundancy is removed.
2. Easier to implement when a new dimension is added to the schema.

3. **Main challenge:** more maintenance effort, because of the additional lookup tables.
4. Queries are slower because more joins are needed.

DESIGN

Fig. 1: Star Schema

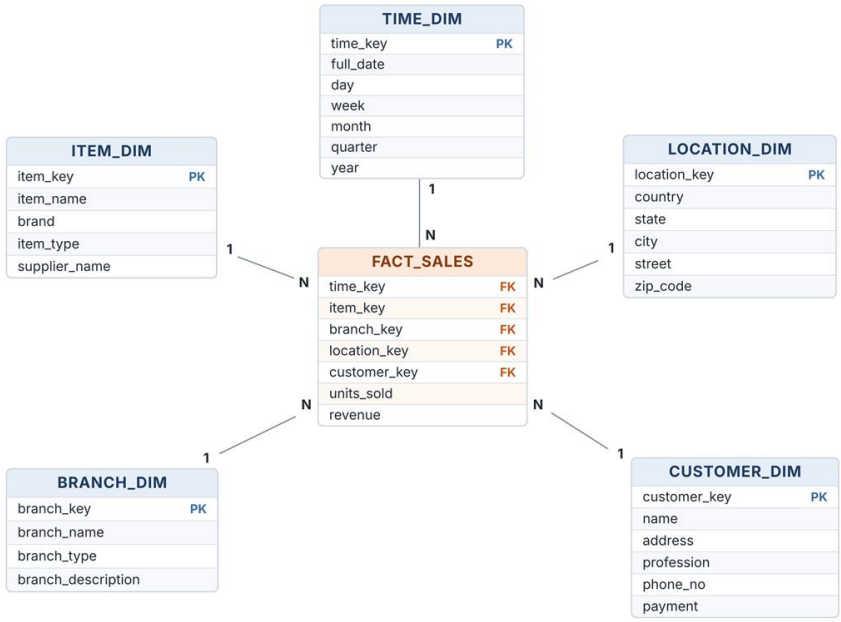
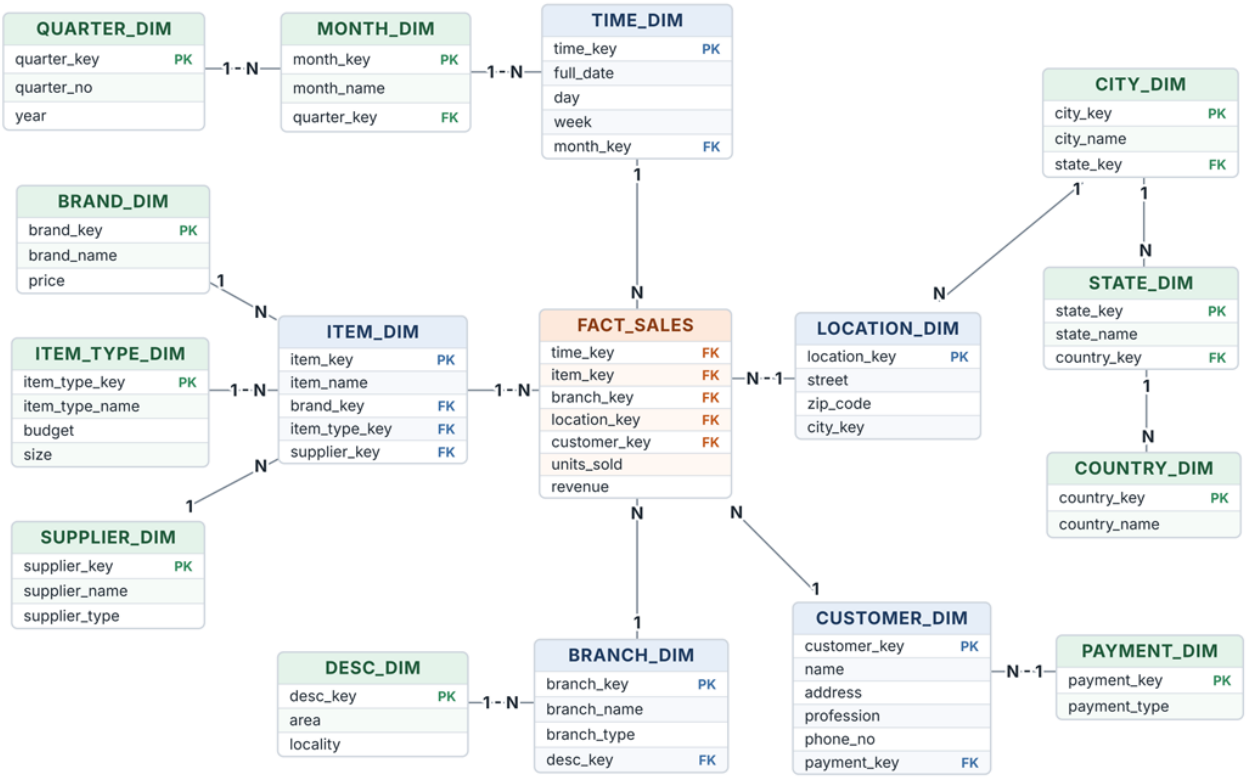


Fig. 2: Snowflake Schema



CONCLUSION

Hence, we have successfully designed the dimensional model of the Star and Snowflake schemas based on the All Electronics sales case study. The star schema gives faster queries with simpler joins, while the snowflake schema saves disk space at the cost of extra joins and maintenance.

EXPERIMENT NO. 02

Implementation of Dimension and Fact Tables

AIM

To implement all dimension tables and the fact table based on the Experiment 1 case study (All Electronics sales data warehouse).

RESOURCES REQUIRED

H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

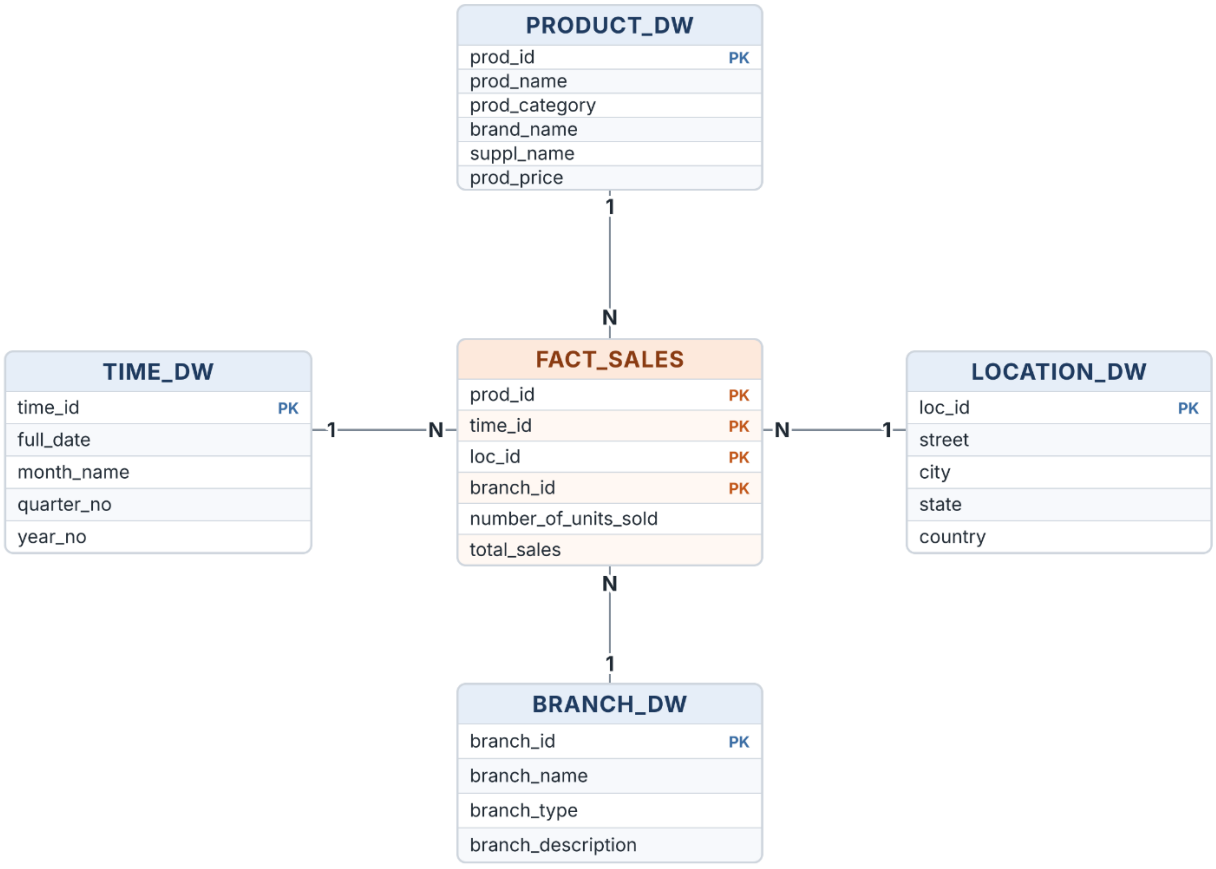
S/W Requirements: Linux or Windows machine, MySQL.

THEORY

All Electronics wants to create a sales data warehouse to keep records of the store's sales with respect to the dimensions time, item (product), branch and location. These dimensions allow the store to keep track of things like the monthly sales of items, and the branches and locations at which the items were sold.

In a star schema, each dimension table has a primary key, and the fact table stores the foreign keys of all dimensions along with the measures (number_of_units_sold, total_sales).

Schema Implemented



IMPLEMENTATION (MySQL)

0. Create and select the database

```
CREATE DATABASE IF NOT EXISTS all_electronics_dw;
USE all_electronics_dw;
```

1. Creating dimension tables

1.1 Product table

```
CREATE TABLE product_dw (
  prod_id       INT PRIMARY KEY,
  prod_name     VARCHAR(60)   NOT NULL,
  prod_category  VARCHAR(255)  NOT NULL,
  brand_name     VARCHAR(255)  NOT NULL,
  suppl_name     VARCHAR(255)  NOT NULL,
  prod_price     DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;
```

1.2 Time table

```
CREATE TABLE time_dw (
  time_id     INT PRIMARY KEY,
  full_date   DATE       NOT NULL,
  month_name  VARCHAR(20) NOT NULL,
  quarter_no  TINYINT     NOT NULL,
  year_no     SMALLINT    NOT NULL
) ENGINE=InnoDB;
```

1.3 Location table

```
CREATE TABLE location_dw (
  loc_id   INT PRIMARY KEY,
  street   VARCHAR(60)  NOT NULL,
  city     VARCHAR(255) NOT NULL,
  state    VARCHAR(255) NOT NULL,
  country  VARCHAR(255) NOT NULL
) ENGINE=InnoDB;
```

1.4 Branch table

```
CREATE TABLE branch_dw (
  branch_id         INT PRIMARY KEY,
  branch_name       VARCHAR(60)  NOT NULL,
  branch_type       VARCHAR(60)  NOT NULL,
  branch_description  VARCHAR(255)
) ENGINE=InnoDB;
```

2. Creating the fact table

```
CREATE TABLE fact_sales (
  prod_id             INT NOT NULL,
  time_id             INT NOT NULL,
  loc_id              INT NOT NULL,
  branch_id           INT NOT NULL,
  number_of_units_sold  INT NOT NULL,
  total_sales         DECIMAL(12,2) NOT NULL,
  PRIMARY KEY (prod_id, time_id, loc_id, branch_id),
  FOREIGN KEY (prod_id)   REFERENCES product_dw(prod_id),
  FOREIGN KEY (time_id)   REFERENCES time_dw(time_id),
  FOREIGN KEY (loc_id)    REFERENCES location_dw(loc_id),
  FOREIGN KEY (branch_id) REFERENCES branch_dw(branch_id)
) ENGINE=InnoDB;
```

3. Inserting values

3.1 Product table

```
INSERT INTO product_dw
  (prod_id, prod_name, prod_category, brand_name, suppl_name, prod_price)
VALUES
(1, 'Rice',   'Grocery',  'Dawat', 'Ramesh', 140),
(2, 'Sugar',  'Grocery',  'Dawat', 'Ramesh',  50),
(3, 'Kurta',  'Clothing', 'Max',   'Lila',   500),
(4, 'Jacket', 'Clothing', 'Max',   'Lila',   700);
```

3.2 Time table

```
INSERT INTO time_dw (time_id, full_date, month_name, quarter_no, year_no) VALUES
(101, '2021-01-07', 'January',  1, 2021),
(102, '2021-02-24', 'February', 1, 2021),
(103, '2021-03-06', 'March',    1, 2021);
```

3.3 Location table

```
INSERT INTO location_dw (loc_id, street, city, state, country) VALUES
(201, 'ML Road', 'Mumbai', 'Maharashtra', 'India'),
(202, 'AL Road', 'Mumbai', 'Maharashtra', 'India');
```

3.4 Branch table

```
INSERT INTO branch_dw
   (branch_id, branch_name, branch_type, branch_description)
VALUES
(301, 'ML Road Branch', 'Retail', 'Retail outlet on ML Road'),
(302, 'AL Road Branch', 'Retail', 'Retail outlet on AL Road');
```

3.5 Fact table

```
INSERT INTO fact_sales
   (prod_id, time_id, loc_id, branch_id, number_of_units_sold, total_sales)
VALUES
(1, 101, 201, 301, 400, 56000.00),    -- Rice:   400 x 140
(4, 102, 202, 302, 500, 350000.00);   -- Jacket: 500 x 700
```

4. Verification

```
SELECT * FROM product_dw;
SELECT * FROM time_dw;
SELECT * FROM location_dw;
SELECT * FROM branch_dw;
SELECT * FROM fact_sales;
```

Combined query (joins the fact table with all dimensions)

```
SELECT p.prod_name, t.month_name, l.city, b.branch_name,
     f.number_of_units_sold AS units_sold, f.total_sales
FROM fact_sales f
JOIN product_dw  p ON f.prod_id   = p.prod_id
JOIN time_dw     t ON f.time_id   = t.time_id
JOIN location_dw l ON f.loc_id    = l.loc_id
JOIN branch_dw   b ON f.branch_id = b.branch_id;
```

Output

prod_name	month_name	city	branch_name	units_sold	total_sales
Rice	January	Mumbai	ML Road Branch	400	56000.00
Jacket	February	Mumbai	AL Road Branch	500	350000.00

Column "units_sold" is number_of_units_sold; "total_sales" is the sale amount.

CONCLUSION

Hence, we have successfully implemented all the dimension tables (product, time, location, branch) and the fact table of our case study using MySQL.

EXPERIMENT NO. 03
OLAP Operations

AIM
To implement the OLAP operations Slice, Dice, Roll-up, Drill-down and Pivot based on the Experiment 1 case study.

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine.

THEORY
OLAP (Online Analytical Processing) lets users analyze multidimensional data interactively. The data is viewed as a data cube whose axes are the dimensions (here Time, Item and Location) and whose cells hold the measure (units sold).

1. Slice
Slice selects one value on one dimension of the cube and produces a sub-cube with one dimension fewer, giving the data from another point of view.

Example: Time = "Q1" turns the 3-D cube into a 2-D table of Location × Item.

2. Dice
Dice selects specific values on two or more dimensions and produces a smaller sub-cube of the same dimensionality.

Example: Location = (Toronto or Vancouver) and Time = (Q1 or Q2) and Item = (mobile or modem). (To locate a single cell, one value is specified for every dimension.)

3. Roll-up
Roll-up performs aggregation on a data cube in either of these ways:

- By climbing up a concept hierarchy for a dimension (e.g. city → country).
- By dimension reduction.

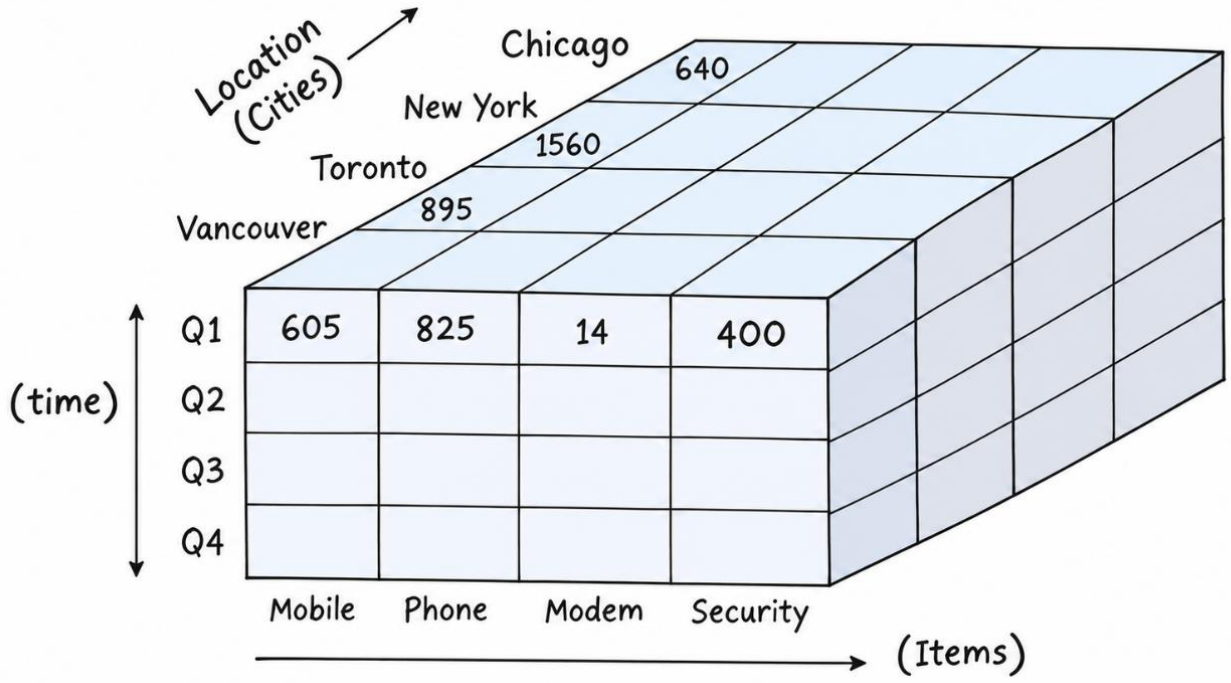
4. Drill-down
Drill-down is the reverse of roll-up. It is performed in either of these ways:

- By stepping down a concept hierarchy for a dimension (e.g. quarter → month).
- By introducing a new dimension.

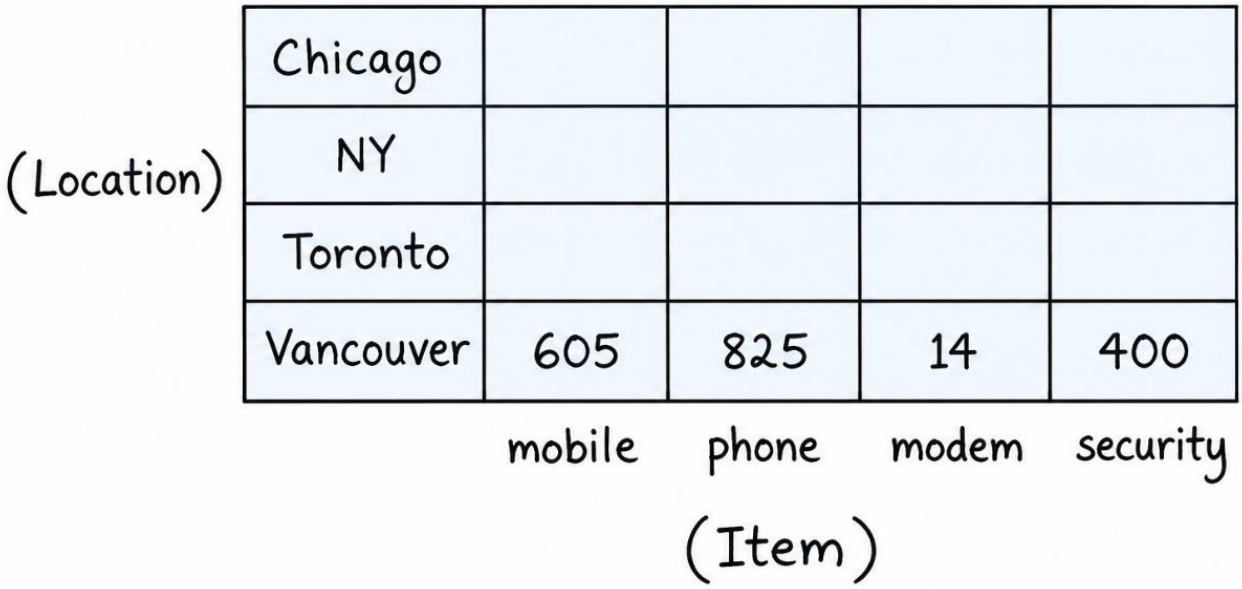
5. Pivot
Pivot (also called rotation) rotates the data axes in the view in order to provide an alternative presentation of the data.

IMPLEMENTATION
Case data (measure = units sold):
Time = Q1 to Q4,
Item = mobile, phone, modem, security,
Location = Vancouver, Toronto, New York, Chicago.
— means no value in the cube for that cell.

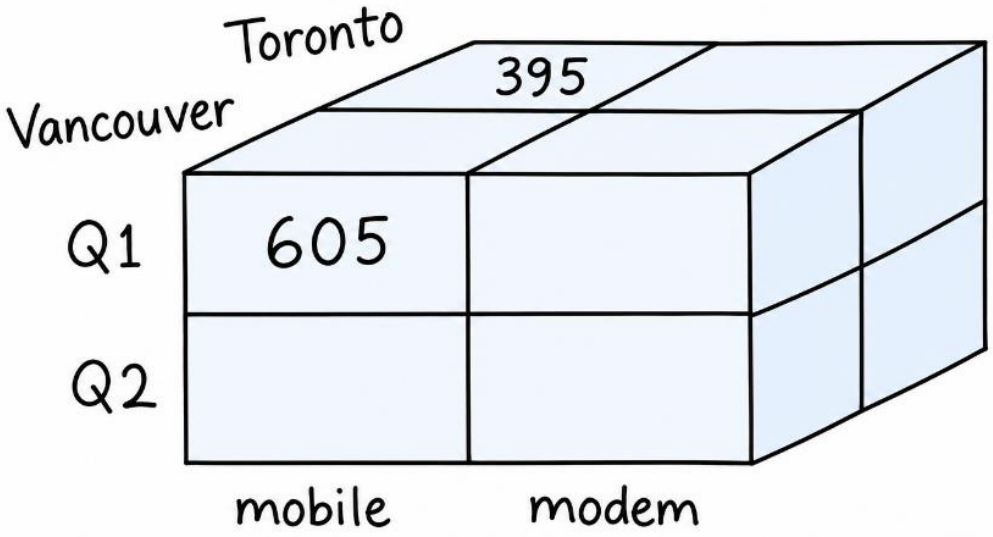
0. Main Cube



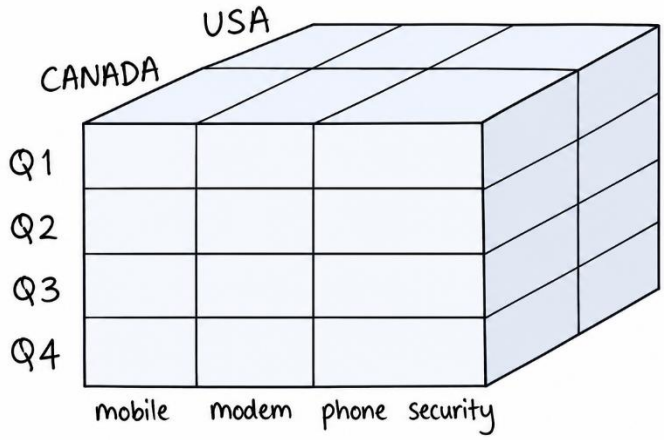
1. Slice: for Time = "Q1"



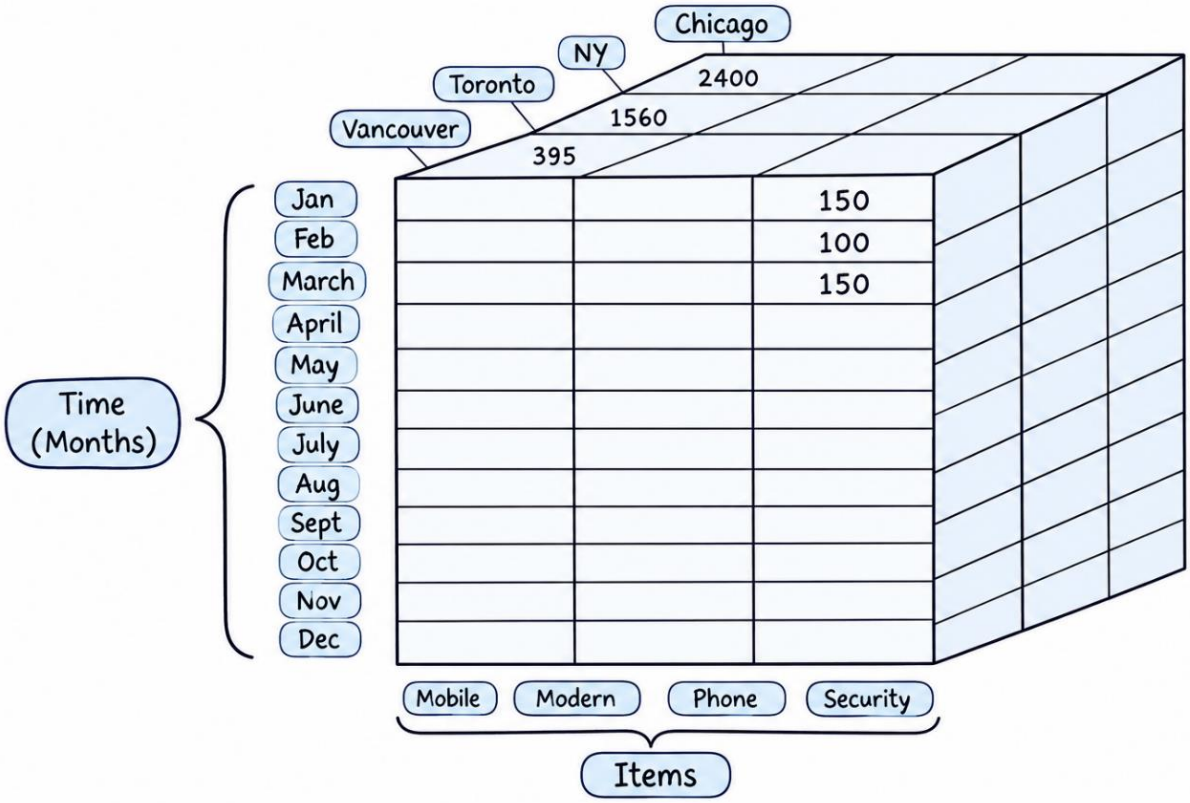
2. Dice: Location (Toronto or Vancouver), Time (Q1 or Q2), Item (mobile or modem)



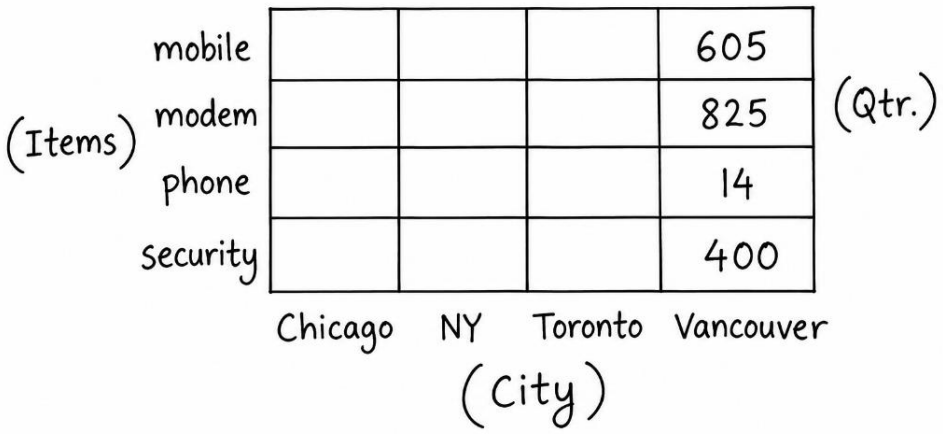
3. Roll-up: Location, city → country



4. Drill-down: Time, quarter → month



5. Pivot: rotate the Q1 slice



CONCLUSION

Hence, we have successfully implemented the OLAP operations on the given case study.

EXPERIMENT NO. 04
Data Discretization and Visualization

AIM:
Implementation of Data Discretization and Visualization.

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine, WEKA Tools.

THEORY:
Data Discretization:-
Data discretization refers to a method of converting a huge number of data values into smaller ones so that the evaluation and management of data become easy. In other words, data discretization is a method of converting attribute values of continuous data into a finite set of intervals with minimum data loss. There are two forms of data discretization: the first is supervised discretization, and the second is unsupervised discretization. Supervised discretization refers to a method in which the class data is used. Unsupervised discretization refers to a method in which the class data is not used. It works on the top-down splitting strategy and bottom-up merging strategy.

Now, we can understand this concept with the help of an example. Suppose we have an attribute of Age with the given values

Age	1,5,9,4,7,11,14,17,13,18,19,31,33,36,42,44,46,70,74,78,77			
Table before Discretization				
Attribute	**Age**	**Age**	**Age**	**Age**
	1,5,4,9,7	11,14,17,13,18,19	31,33,36,42,44,46	70,74,77,78
After Discretization	Child	Young	Mature	Old
Table after Discretization				

Data Visualization:-
Data visualization is the process of translating large data sets and metrics into charts, graphs and other visuals. The resulting visual representation of data makes it easier to identify and share real-time trends, outliers, and new insights about the information represented in the data.

A dashboard is an information visualization tool. It helps you monitor events or activities at a glance by providing insights on one or more pages or screens. Unlike an infographic, which presents a static graphical representation, a dashboard conveys real-time information by pulling complex data points directly from large data sets. An interactive dashboard makes it easy to sort, filter, or drill into different types of data as needed. Data science techniques can be used to identify what is happening, why it's happening, and what will happen next at speed.

As the amount of big data increases, more people are using data visualization tools to access insights on their computer and on mobile devices. Dashboards are used by business people, data analysts, and data scientists to make data-driven business decisions.

CONCLUSION:
Hence, we have successfully implemented Data Visualization and Discretization on Weather dataset and Iris dataset using WEKA Tools.

EXPERIMENT NO. 05
Bayesian Algorithm

AIM:
Implementation of Bayesian Algorithm.

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine, WEKA Tools, Jupyter Notebook, Python.

THEORY:

Naïve Bayes:

Naive Bayes methods are a set of supervised learning algorithms based on applying Bayes' theorem with the "naive" assumption of conditional independence between every pair of features given the value of the class variable. Bayes' theorem states the following relationship, given class variable y and dependent feature vector x_1 through $x□$:

$$P(y|x_1, \dots, x□) = \frac{P(y)P(x_1, \dots, x□|y)}{P(x_1, \dots, x□)}$$

Using the naive conditional independence assumption that

$$P(y|x_1, \dots, x□) = \frac{P(y)P(x_1, \dots, x□|y)}{P(x_1, \dots, x□)}$$

For all I, this relationship is simplified to

$$P(y \mid x_1, \dots, x□) = [\, P(y) \cdot \prod_{i=1}^{n} P(x_i \mid y) \,] / P(x_1, \dots, x□)$$

Since $P(x1, \dots, xn)$ is constant given the input, we can use the following classification rule:

$$P(y \mid x_1, \dots, x□) \propto P(y) \prod_{i=1}^{n} P(x_i \mid y) \Rightarrow \hat{y} = argmax_y \, (P(y) \prod_{i=1}^{n} P(x_i \mid y))$$

And we can use Maximum a Posteriori (MAP) estimation to estimate $P(y)$ and $P(x_i \mid y)$, the former is then the relative frequency of class y in the training set.

The different naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of $P(x_i \mid y)$.

In spite of their apparently over-simplified assumptions, naive Bayes classifiers have worked quite well in many real-world situations, famously document classification and spam filtering. They require a small amount of training data to estimate the necessary parameters. (For theoretical reasons why naive Bayes works well, and on which types of data it does, see the references below.)

Naive Bayes learners and classifiers can be extremely fast compared to more sophisticated methods. The decoupling of the class conditional feature distributions means that each distribution can be independently estimated as a one-dimensional distribution. This in turn helps to alleviate problems stemming from the curse of dimensionality.

Gaussian Naive Bayes:
When working with continuous data, an assumption often taken is that the continuous values associated with each class are distributed according to a normal (or Gaussian) distribution. The likelihood of the features is assumed to be:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} e^{-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}}$$

Sometimes we assume variance

- is independent of Y (i.e., σi),
- or independent of Xi (i.e., σk)
- or both (i.e., σ)

Gaussian Naive Bayes supports continuous valued features and models each as conforming to a Gaussian (normal) distribution.

Code to Implement Using Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load Iris dataset
data = load_iris()
X = data.data
y = data.target

# Create DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df["class"] = y

print("First five rows:")
print(df.head())

# Convert class numbers to names
df["class"] = df["class"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

print("\nData after converting class names:")
print(df.head())

# Group by class and display statistics
print("\nStatistics:")
print(df.groupby("class").describe())

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
   X, y,
   train_size=0.5,
   random_state=0
)

print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)

# Create Naive Bayes model
nb = GaussianNB()
```

```
# Train the model
model = nb.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)
print("\nPredicted values:")
print(y_pred)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(f"{accuracy * 100}%")

# Classification Report
print("\nClassification Report:")
print(
   classification_report(
      y_test,
      y_pred,
      target_names=data.target_names
   )
)

# Display confusion matrix
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xticks([0, 1, 2], data.target_names)
plt.yticks([0, 1, 2], data.target_names)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

Output:-

```
First five rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  class
0                5.1               3.5  ...               0.2      0
1                4.9               3.0  ...               0.2      0
2                4.7               3.2  ...               0.2      0
3                4.6               3.1  ...               0.2      0
4                5.0               3.6  ...               0.2      0
[5 rows x 5 columns]

Data after converting class names:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  class
0                5.1               3.5  ...               0.2  setosa
1                4.9               3.0  ...               0.2  setosa
2                4.7               3.2  ...               0.2  setosa
```

```
3                4.6                3.1                ...  0.2                setosa
4                5.0                3.6                ...  0.2                setosa
[5 rows x 5 columns]

Statistics:
        sepal length (cm)                                  ...
                   count  mean       std       min       ...  25%  50%  75%  max
class                                                      ...
setosa              50.0  5.006  0.352490       4.3       ...  0.2  0.2  0.3  0.6
versicolor          50.0  5.936  0.516171       4.9       ...  1.2  1.3  1.5  1.8
virginica           50.0  6.588  0.635880       4.9       ...  1.8  2.0  2.3  2.5
[3 rows x 32 columns]

Training data: (75, 4)
Testing data: (75, 4)

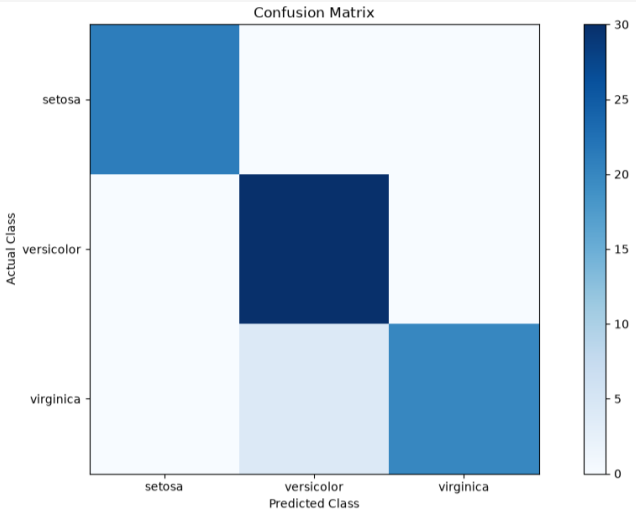
Predicted values:
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 1 1 1 2 0 2 0 0 1 2 2 1 2 1 2 1 1 2 1 1 2 1 2 1 0 2 1 1 1 1 2 0 0 2 1 0 0 1]

Confusion Matrix:
[[21  00  00]
 [00  30  00]
 [00  04  20]]

Accuracy:
0.9466666666666667

Accuracy Percentage:
94.66666666666667%

Classification Report:
              Precision Recall F1-score Support
setosa        1.00      1.00   1.00     21
versicolor    0.88      1.00   0.94     30
virginica     1.00      0.83   0.91     24
accuracy                       0.95     75
macro avg     0.96      0.94   0.95     75
weighted avg  0.95      0.95   0.95     75
```



CONCLUSION:

Hence, we have successfully implemented Bayes Algorithm for Classification on iris dataset using Python and WEKA Tools.

EXPERIMENT NO. 06
Data Pre-processing, Classification, Clustering and Association

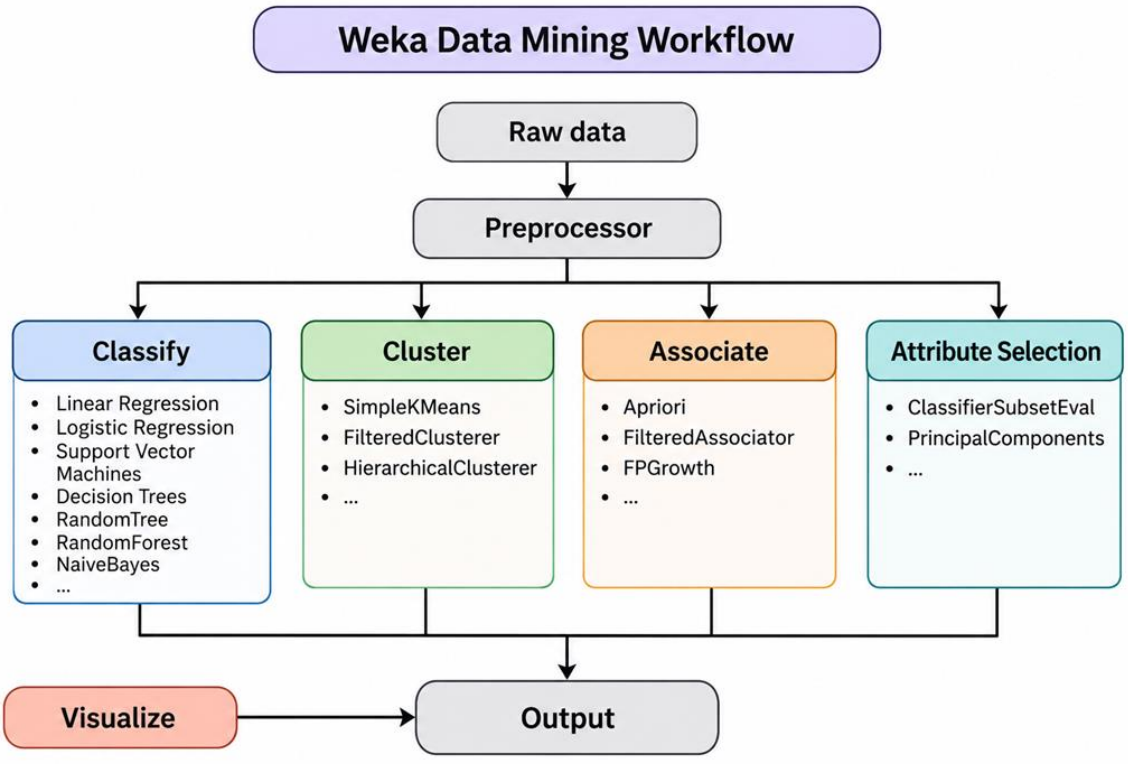
AIM:
Perform data pre-processing task and demonstrate performing classification, clustering, association algorithm on data sets using data mining tool (WEKA/R tool).

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine, WEKA Tools.

THEORY:
Weka is a collection of machine learning algorithms for data mining tasks. The algorithms can directly be applied to a dataset or called from your own java code. Weka contains tools for data pre-processing, classification, regression, clustering, association rules, visualization.

WEKA, an open source software, provides tools for data pre-processing, implementation of several Machine Learning algorithms, and visualization tools so that you can develop machine learning techniques and apply them to real world data mining problems. What WEKA offers is summarized in the following diagram:



Datasets for WEKA are formatted according to the ARFF format. An ARFF (Attribute-Relation File Format) file is an ASCII text file that describes a list of instances sharing a set of attributes. ARFF files have two distinct sections. The first section is the Header information, which is followed by the Data information.

The ARFF Header Section:-
The ARFF Header section of the file contains the relation declaration and attributes declarations.

i) The @relation Declaration:-
The relation name is defined as the first line in the ARFF file. The format is:

> @relation <relation-name> where <relation-name> is a string.

ii) The @attribute Declarations:-

Attribute declarations take the form of an ordered sequence of @attribute statements. Each attribute in the data set has its own @attribute statement which uniquely defines the name of that attribute and its data type. The format for the @attribute statement is:

```
@attribute <attribute-name> <datatype>
```

Where the <attribute-name> must start with an alphabetic character. If spaces are to be included in the name then the entire name must be quoted. Example header on the standard IRIS dataset looks like this:

```
% 1. Title: Iris Plants Database
%
@RELATION iris
@ATTRIBUTE sepallength NUMERIC
@ATTRIBUTE sepalwidth NUMERIC
@ATTRIBUTE petallength NUMERIC
@ATTRIBUTE petalwidth NUMERIC
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
```

iii) The @data Declaration:-

The @data declaration is a single line denoting the start of the data segment in the file. The format is: @data

The <datatype> can be any of the four types currently (version 3.2.1) supported by Weka:

- numeric
- <nominal-specification>
- string
- date [<date-format>]

Where <nominal-specification> and <date-format> are defined below. The keywords numeric, string and date are case insensitive.

1. Numeric attributes:

Numeric attributes can be real or integer numbers.

2. Nominal attributes:

Nominal values are defined by providing an <nominal-specification> listing the possible values: {<nominal-name1>, <nominal-name2>, <nominal-name3>,...} For example, the class value of the Iris dataset can be defined as follows:

```
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
```

Values that contain spaces must be quoted.

3. String attributes:

String attributes allow us to create attributes containing arbitrary textual values. This is very useful in text-mining applications, as we can create datasets with string attributes, then write Weka Filters to manipulate strings (like StringToWordVectorFilter). String attributes are declared as follows:

```
@ATTRIBUTE LCC string
```

4. Date attributes:

Date attribute declarations take the form:

```
@attribute <name> date [<date-format>]
```

where <name> is the name for the attribute and <date-format> is an optional string specifying how date values should be parsed and printed (this is the same format used by SimpleDateFormat). The default format string accepts the ISO-8601 combined date and time format: "yyyy-MM-dd'T'HH:mm:ss". Dates must be

specified in the data section using the string representation specified in the attribute declaration. For example:

```
@RELATION Timestamps
@ATTRIBUTE timestamp DATE "yyyy-MM-dd HH:mm:ss"
@DATA
"2001-04-03 12:12:12"
"2001-05-03 12:59:55"
```

The Data of the ARFF file looks like the following: @DATA

```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
5.4,3.9,1.7,0.4,Iris-setosa
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
```

Lines that begin with a % are comments. The @RELATION, @ATTRIBUTE and @DATA declarations are case insensitive.

CONCLUSION:

Hence, we have successfully performed the data pre-processing task and demonstrated classification, clustering, association algorithms and visualization on dataset using data mining tools.

EXPERIMENT NO. 07
K-Means Clustering

AIM:
Implementation of clustering algorithm (K-means/K-Medoids).

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

S/W Requirements: Linux or Windows machine, Jupyter Notebook, Python.

THEORY:
Clustering:-
Clustering is the task of dividing the population or data points into a number of groups such that data points in the same group are more similar to other data points in the same group than those in other groups. In simple words, the aim is to segregate groups with similar traits and assign them into clusters.

Let's understand this with an example. Suppose, you are the head of a rental store and wish to understand preferences of your customers to scale up your business. Is it possible for you to look at details of each customer and devise a unique business strategy for each one of them? Definitely not. But, what you can do is to cluster all of your customers into say 10 groups based on their purchasing habits and use a separate strategy for customers in each of these 10 groups. And this is what we call clustering.

k-means clustering:-
k-means clustering is a method of vector quantization, originally from signal processing, that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid), serving as a prototype of the cluster. This results in a partitioning of the data space into Voronoi cells. k-means clustering minimizes within-cluster variances (squared Euclidean distances), but not regular Euclidean distances, which would be the more difficult Weber problem: the mean optimizes squared errors, whereas only the geometric median minimizes Euclidean distances. For instance, better Euclidean solutions can be found using k-medians and k-medoids.

The k-means algorithm takes the input parameter, k, and partitions a set of n objects into k clusters so that the resulting intra-cluster similarity is high but the inter-cluster similarity is low. Cluster similarity is measured in regard to the mean value of the objects in a cluster, which can be viewed as the cluster's centroid or center of gravity.

Euclidean Distance:-
In many applications, and in particular when comparing distances, it may be more convenient to omit the final square root in the calculation of Euclidean distances. The value resulting from this omission is the square of the Euclidean distance, and is called the squared Euclidean distance. As an equation, it can be expressed as a sum of squares:

$$E = \sum_{i=1}^{k} \sum_{p \in C_i} |p - m_i|^2$$

Steps for k-means clustering:-
STEP-1: Choose the number of clusters k.

STEP-2: Select k random points within the data range as centroids.

STEP-3: Assign all the points to the closest cluster centroid.

STEP-4: Recompute the centroids of newly formed clusters.

STEP-5: Repeat steps 3 and 4.

Stopping criteria:-

There are essentially three stopping criteria that can be adopted to stop the K-means algorithm:

1. Centroids of newly formed clusters do not change
2. Points remain in the same cluster
3. Maximum number of iterations are reached

We can stop the algorithm if the centroids of newly formed clusters are not changing. Even after multiple iterations, if we are getting the same centroids for all the clusters, we can say that the algorithm is not learning any new pattern and it is a sign to stop the training.

Another clear sign that we should stop the training process if the points remain in the same cluster even after training the algorithm for multiple iterations.

Finally, we can stop the training if the maximum number of iterations is reached. Suppose if we have set the number of iterations as 100. The process will repeat for 100 iterations before stopping.

Implementation: -

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
# 1. Initialize Centroids
def init_centroids(k, X):
    centroids = []
    for i in range(k):
        x1 = np.random.uniform(
            np.min(X[:, 0]),
            np.max(X[:, 0])
        )
        x2 = np.random.uniform(
            np.min(X[:, 1]),
            np.max(X[:, 1])
        )
        centroids.append([x1, x2])
    return np.array(centroids)
# 2. Calculate Euclidean Distance
def distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))
# 3. Assign Each Point to Nearest Cluster
def assign_cluster(k, X, centroids):
    cluster = []
    for i in range(len(X)):
        distances = []
        for j in range(k):
            d = distance(X[i], centroids[j])
            distances.append(d)
        nearest = np.argmin(distances)
        cluster.append(nearest)
    return np.array(cluster)
# 4. Calculate New Centroids
def compute_centroids(k, X, cluster):
    new_centroids = []
```

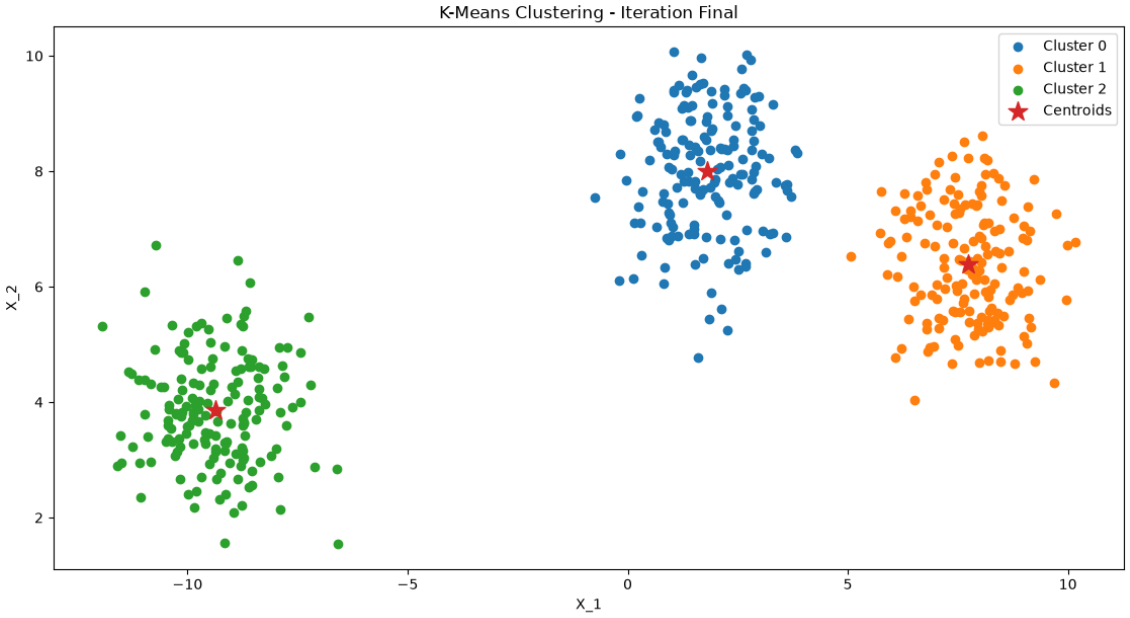
```
    for i in range(k):
        points = []
        for j in range(len(X)):
            if cluster[j] == i:
                points.append(X[j])
        new_centroids.append(np.mean(points, axis=0))
    return np.array(new_centroids)
# 5. Calculate Change in Centroids
def measure_change(old_centroids, new_centroids):
    change = 0
    for old, new in zip(old_centroids, new_centroids):
        change += distance(old, new)
    return change
# 6. Display Clusters
def show_clusters(X, cluster, centroids, iteration):
    plt.figure(figsize=(8, 6))
    # Display each cluster
    for i in range(3):
        plt.scatter(
            X[cluster == i, 0],
            X[cluster == i, 1],
            label="Cluster " + str(i)
        )
    # Display centroids
    plt.scatter(
        centroids[:, 0],
        centroids[:, 1],
        marker="*",
        s=200,
        label="Centroids"
    )
    plt.xlabel("X_1")
    plt.ylabel("X_2")
    plt.title("K-Means Clustering - Iteration " + str(iteration))
    plt.legend()
    plt.show()
# 7. K-Means Algorithm
def k_means(k, X):
    # Step 1: Initialize centroids
    centroids = init_centroids(k, X)
    change = 100
    iteration = 1
    while change > 0.001:
        # Step 2: Assign points to nearest centroid
        cluster = assign_cluster(k, X, centroids)
        # Step 3: Calculate new centroids
        new_centroids = compute_centroids( k,X,cluster)
        # Step 4: Calculate centroid movement
        change = measure_change(centroids,new_centroids)
        print("\nIteration:", iteration)
        print("New Centroids:")
```

```
        print(new_centroids)
        print("Centroid Change:", change)
        # Update centroids
        centroids = new_centroids
        iteration += 1
    return cluster, centroids
# 8. Generate Dataset
np.random.seed(20)
X_train, _ = make_blobs(
    n_samples=500,
    centers=3,
    n_features=2,
    random_state=20
)
# 9. Apply K-Means
k = 3
cluster, centroids = k_means(k, X_train)
# 10. Display Final Result
show_clusters (X_train, cluster, centroids, "Final")
```

OUTPUT:

```
Iteration: 1
New Centroids:
[[ 1.80025806  7.99455026]
 [ 7.74173571  6.38572168]
 [-9.3753175   3.85672799]]
Centroid Change: 7.535172335516927

Iteration: 2
New Centroids:
[[ 1.80025806  7.99455026]
 [ 7.74173571  6.38572168]
 [-9.3753175   3.85672799]]
Centroid Change: 0.0
```



CONCLUSION:

Hence, we have successfully implemented clustering algorithm i.e., k-means algorithm using Python.

EXPERIMENT NO. 08
Hierarchical Clustering

AIM:
Implementation of any one Hierarchical Clustering method.

RESOURCES REQUIRED:
H/W Requirements: P I and above, RAM 128MB, Printers and Cartridges.

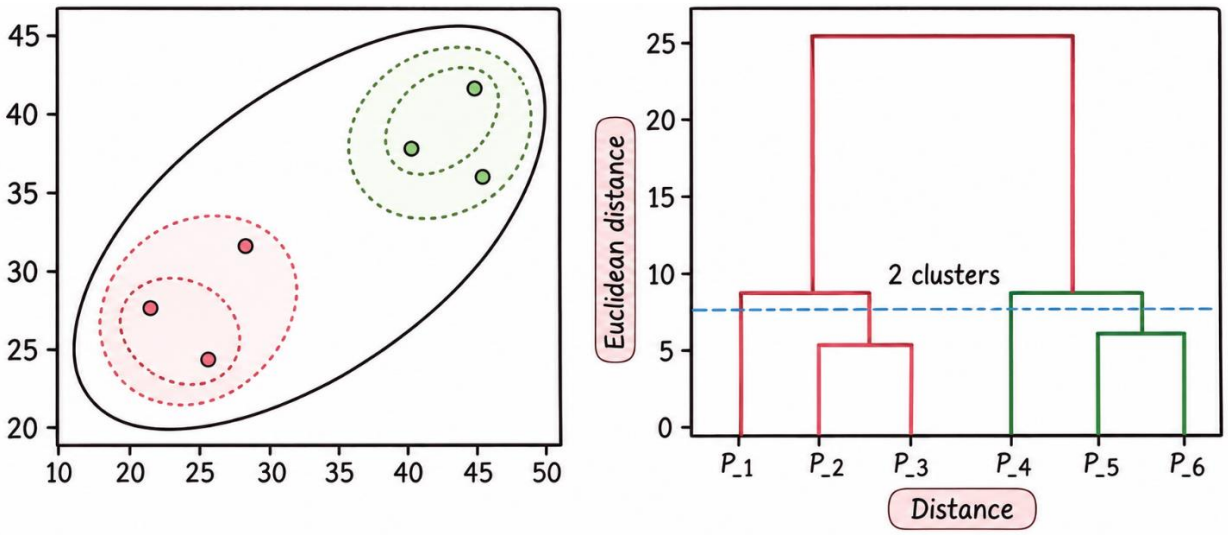
S/W Requirements: Linux or Windows machine, WEKA Tools.

THEORY:

Hierarchical Clustering:-
Hierarchical clustering is another unsupervised machine learning algorithm, which is used to group the unlabeled datasets into clusters and is also known as hierarchical cluster analysis or HCA.

In this algorithm, we develop the hierarchy of clusters in the form of a tree, and this tree-shaped structure is known as the dendrogram. Sometimes the results of K-means clustering and hierarchical clustering may look similar, but they both differ depending on how they work, as there is no requirement to predetermine the number of clusters as we did in the K-Means algorithm.

Working on Dendrogram in Hierarchical Clustering:-
The dendrogram is a tree-like structure that is mainly used to store each step as a memory that the HC algorithm performs. In the dendrogram plot, the Y-axis shows the Euclidean distances between the data points, and the x-axis shows all the data points of the given dataset.

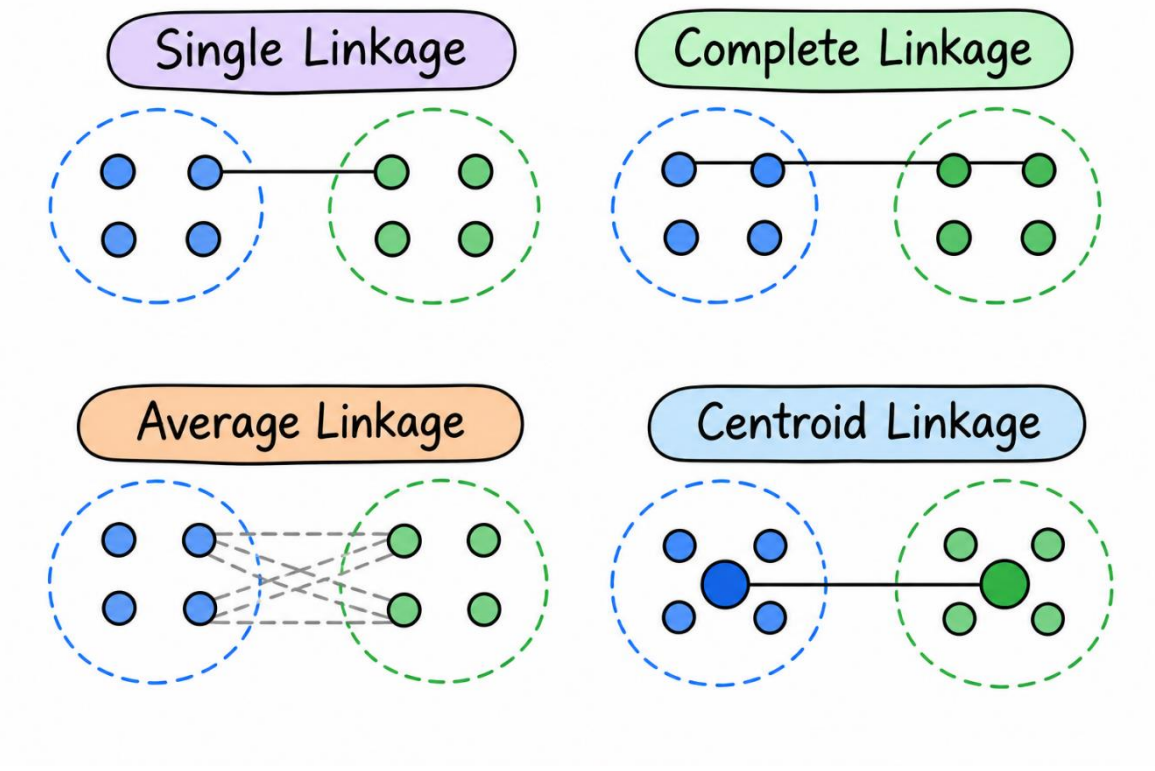


In the above diagram, the left part is showing how clusters are created in agglomerative clustering, and the right part is showing the corresponding dendrogram.

Measure for the distance between two clusters:-
As we have seen, the closest distance between the two clusters is crucial for the hierarchical clustering. There are various ways to calculate the distance between two clusters, and these ways decide the rule for clustering. These measures are called Linkage methods. Some of the popular linkage methods are given below:

1. **Single Linkage:** It is the shortest distance between the closest points of the clusters.
2. **Complete Linkage:** It is the farthest distance between the two points of two different clusters. It is one of the popular linkage methods as it forms tighter clusters than single-linkage.

3. **Average Linkage:** It is the linkage method in which the distance between each pair of the dataset is added up and then divided by the total number of pairs to calculate the average distance between two clusters. It is also one of the most popular linkage methods.
4. **Centroid Linkage:** It is the linkage method in which the distance between the centroid of the clusters is calculated.



CONCLUSION:

Hence, we have successfully implemented Hierarchical Clustering on iris dataset using WEKA Tools.

EXPERIMENT NO. 09
Data Exploration and Data Visualization Using Power BI

AIM:
Case Study on Data Exploration and Data Visualization Using Power BI

THEORY:
Case Study: Supermarket Sales Analysis

Introduction
Data Exploration is the process of examining data to understand its structure, quality, patterns and relationships.

Data Visualization represents data graphically using charts, graphs and dashboards. It helps users understand complex data easily.

Power BI is a business intelligence tool used to analyze data and create interactive visualizations and dashboards.

Problem Statement
A supermarket wants to analyze its sales data to understand:

- Which product category has the highest sales?
- How do sales change over time?
- What is the contribution of each category?
- Is there a relationship between quantity sold and sales?

Sample Dataset

Invoice	Category	City	Quantity	Sales (₹)	Profit (₹)
1001	Electronics	Mumbai	5	25,000	4,000
1002	Grocery	Pune	10	12,000	2,500
1003	Clothing	Mumbai	7	18,000	3,000
1004	Electronics	Nashik	3	15,000	2,000
1005	Grocery	Pune	8	10,000	2,200
1006	Clothing	Nashik	6	14,000	2,400

Data Types
- **Categorical:** Category, City
- **Numerical:** Quantity, Sales, Profit
- **Identifier:** Invoice

Data Exploration
The following activities are performed during data exploration:

- Check number of rows and columns.
- Identify data types.
- Check missing values.
- Check duplicate records.

- Identify incorrect/inconsistent values.
- Calculate minimum, maximum and average values.
- Identify important patterns and relationships.

Data Preparation

Before visualization:

1. Remove duplicate records.
2. Handle missing values.
3. Correct data types.

Data Visualization Using Power BI

Analysis	Visualization	Purpose
Sales by category	Bar Chart	Compare categories
Sales by city	Column Chart	Compare cities
Monthly sales	Line Chart	Identify trends
Quantity vs Sales	Scatter Plot	Identify relationship

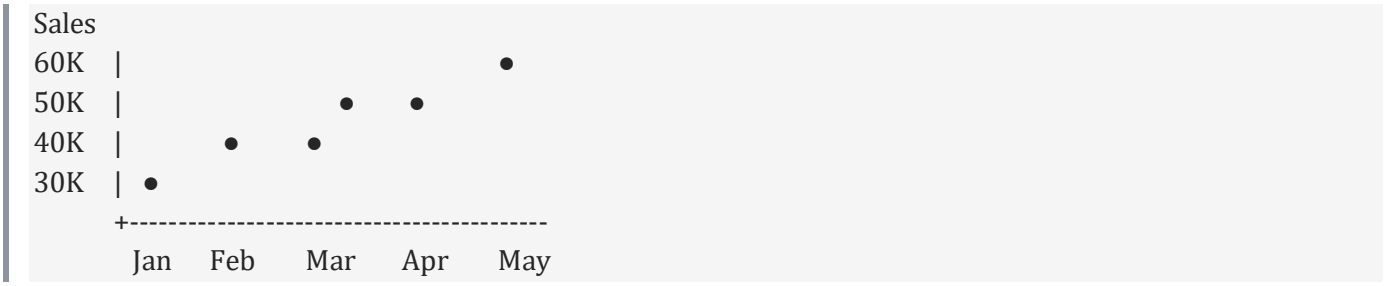
Example Charts

A. Sales by Category – Bar Chart



Interpretation: Electronics has the highest sales in this illustrative example.

B. Monthly Sales – Line Chart



Interpretation: The line chart helps identify increases and decreases in sales over time.

C. Quantity vs Sales – Scatter Plot

Purpose: To examine whether quantity sold and sales amount have a relationship.
Interpretation: A scatter plot helps identify whether the two numerical variables show a positive, negative or weak relationship.

Proposed Power BI Dashboard

SUPERMARKET SALES DASHBOARD			
TOTAL SALES ₹94,000	TOTAL PROFIT ₹16,100	TOTAL QTY 39 Units	NO. OF RECORDS 6
SALES BY PRODUCT CATEGORY		Electronics / Grocery / Clothing	
SALES BY CITY Mumbai vs Pune		QUANTITY BY CATEGORY Electronics/Grocery/ Clothing	
SALES vs PROFIT (Scatter Chart)			

Findings

From the sample illustrative analysis:

1. Electronics has the highest sales.
2. Sales can be compared effectively across different cities.
3. Monthly visualization helps identify sales trends.
4. Scatter plots help examine relationships between numerical attributes.

Advantages of Power BI

- Easy-to-understand visualizations
- Interactive dashboards
- Supports multiple chart types
- Helps identify trends and patterns
- Provides filtering and drill-down
- Useful for data analysis and decision-making

CONCLUSION:

Data exploration and data visualization are important activities in the data mining process.

Data exploration helps understand the structure, quality and characteristics of data, while visualization helps represent the discovered information in an understandable form. The proposed dashboard uses visualizations to provide a summarized view of sales performance.

Through such visualization, users can identify comparisons, trends, proportions and relationships in the dataset.